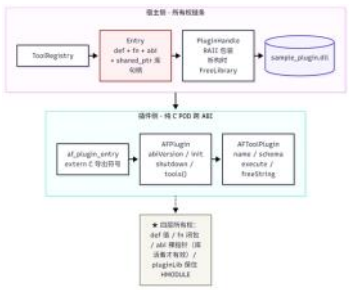
L4-v1

2026年10月12日 16:24

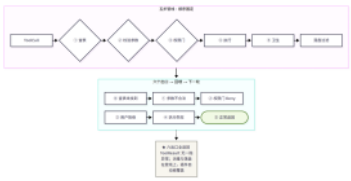
1. 新增类图



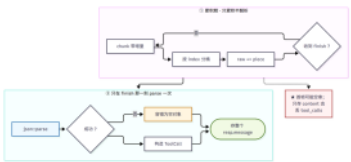
2. 生命周期



3. 2典调用

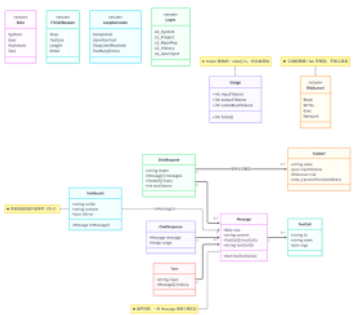


4. 2典参数传递



5. 3种变更.

```
                                code/
├── types.hpp                 ← 契约层 (L2 定稿, L3 零改动, 仍零改动)
├── model_provider.hpp        ← 端口① (L2 定稿, 零改动)
├── http_transport.hpp        ← 端口② (零改动)
├── http_winhttp.cpp          ← 端口②的 Windows 实现 (L3 沿用)
├── sse_parser.{hpp,cpp}      ← 字节流 → 事件 (L3 沿用, 一行没改)
├── sandbox.hpp               ← 端口③: 仍然只有声明 (待实现)
├── tool_registry.hpp         ← ★ 只加不改: +loadPlugin(), Entry 加两个
字段
├── tool_registry.cpp         ← ★ 新写: 注册 · 校验 · 权限 · 派发 · 输出
过滤
├── plugin.{hpp,cpp}          ← ★ 新写: C ABI + LoadLibraryW + RAII
├── providers.cpp             ← ★ 改写: tool_calls 累积 + tools 下发
├── main.cpp                  ← ★ 改写: v0 → v1 (硬编码的 for 循环)
├── plugins/
│   ├── sample_plugin.cpp     ← ★ 可编译并加载的 WordCount 插件
│   └── build_plugin.ps1      ← ★ 编成 .dll + 校验导出符号名
└── mock_server.py            ← 开始真的发 tool_calls
```



2典调用完整流程:

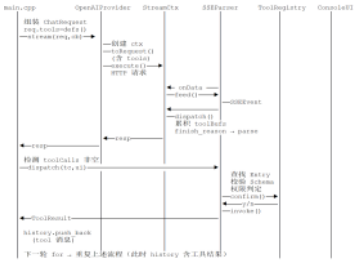
```
阶段 1: 请求侧 —— 工具定义下发
main.cpp → ChatRequest{tools} → OpenAIProvider → JSON → HTTP

阶段 2: 响应侧 —— 工具调用增量累积
字节流 → SSEParser → SSEEvent → StreamCtx.toolBufs → 一次 parse
→ ChatResponse{message.toolCalls}

阶段 3: 派发侧 —— 五步管线执行工具
main → registry.dispatch → 查找/校验/权限/执行/过滤 → ToolResult
→ history.push_back{tool 消息}

阶段 4: 回喂侧 —— 下一轮把工具结果发回模型
history 里已有 assistant{tool_calls} + tool{结果} → toRequest
→ 模型看到结果 → 给出最终回答 → toolCalls 为空 → 任务结束
```

6



分步详解

阶段 1：请求侧 —— 工具定义下发

步骤 1.1：main.cpp 组装 ChatRequest

```
for (int round = 0; round < kMaxRounds; ++round) {
    ChatRequest req;
    req.system   = "你是一个编码 Agent。原则：\n"
                   "1. 需要文件内容时必须**调用工具去读**，不要凭猜测回答。\n"
                   "2. 工具报错是信息不是失败 —— 读它，然后换方法。\n"
                   "3. 回答要给出依据（哪个文件、哪几行）。\n";
    req.messages = history;
    req.tools    = registry.defs();    // ★ 关键：下发工具定义
    ...
}
```

- req.tools = registry.defs(); 是必须的。不下发工具，模型永远不会调工具——它根本不知道有哪些工具可用。
- registry.defs() 返回的 ToolDef 数组顺序必须稳定，因为 OpenAI 的 prompt caching 是前缀缓存：tools 数组顺序一变，整个前缀缓存失效（NFR-2）。

步骤 1.2：OpenAIProvider::stream 创建 StreamCtx

```
ChatResponse stream(const ChatRequest& req,
                    const std::function<bool(const ChatChunk&)>& cb) override {
    StreamCtx ctx;
    ctx.onChunk = cb;
    return run(req, &ctx);
}
```

- StreamCtx 是栈对象，生命周期覆盖整个 stream 调用。
- ctx.onChunk = cb 保存了 main.cpp 传入的 lambda（捕获了 Renderer）。

步骤 1.3：run 调用 toRequest 序列化

```
ChatResponse run(const ChatRequest& req, StreamCtx* sctx) {
    json body = toRequest(req, sctx != nullptr);
    if (sctx) body["stream_options"] = {{"include_usage", true}};
    ...
}
```

- stream_options: {include_usage: true} 是流式请求必须显式要的。默认不带 usage，成本就没法统计。

步骤 1.4：toRequest 把 tools 序列化成 OpenAI 格式

```
if (!req.tools.empty()) {
    json tools = json::array();
    for (const auto& t : req.tools) {
        tools.push_back({{"type", "function"},
                         {"function", {{"name", t.name},
                                       {"description", t.description},
                                       {"parameters", t.inputSchema}}}});
    }
    j["tools"] = tools;
    switch (req.toolChoice) {
        case ToolChoice::Auto:     j["tool_choice"] = "auto"; break;
        case ToolChoice::None:     j["tool_choice"] = "none"; break;
        case ToolChoice::Required: j["tool_choice"] = "required"; break;
        case ToolChoice::Named:
            j["tool_choice"] = {{"type", "function"},
                                {"function", {{"name", req.forcedToolName.value_or("")}}}};
            break;
    }
}
```

```
请求体（发给 API 的 JSON）
├─ stream: true                          → 开启流式
├─ stream_options: { include_usage }     → 流式选项（请求侧）
├─ tool_choice: "auto"|"none"|...        → 工具调用策略（请求侧）
└─ tools: [...]                          → 工具定义列表
```

```
{
  "choices": [],
  "usage": {
    "prompt_tokens": 42,
    "completion_tokens": 128,
    "total_tokens": 170
  }
}
```

- OpenAI 的工具格式是嵌套的：{type: "function", function: {name, description, parameters}}。
- tool_choice 有四种模式：auto（模型自选）、none（禁止调用）、required（必须调）、{type, function}（强制调指定工具）。
- 不做任何排序——顺序由 registry.defs() 保证。

步骤 1.5：execute 发送 HTTP 请求

```
auto r = execute(cfg_.baseUrl + "/chat/completions", body.dump(), headers, sctx);
```

- body.dump() 把 JSON 序列化成字符串。
- sctx 非空表示流式请求，execute 内部走 transport_->postStream(spec, lambda)。

阶段 2：响应侧 —— 工具调用增量累积

步骤 2.1：传输层逐块回调

```
// WinHttpTransport::doRequest 里
while (WinHttpReadData(req, buf, sizeof(buf), &read) && read > 0) {
    if (onData && !onData(buf, size_t(read))) { cancelled = true; break; }
}
```

- buf 是 8KB 栈缓冲，每次 WinHttpReadData 填充多少就是多少。
- TCP 任意切分 → HTTP 任意分块 → onData 收到的字节不保证对齐任何边界。

步骤 2.2：StreamCtx::feed 驱动 SSEParser

```
bool feed(const char* data, std::size_t len) {
    for (auto& ev : parser.push(std::string_view(data, len))) {
        if (ev.isDone()) return false;
        if (!dispatch(ev)) return false;
    }
```

```
    return true;
}
```

- parser.push 内部 buffer_.append(chunk) 立即拷贝，所以 buf 被复用不影响。
- parser.push 返回本次能完整切出的所有事件（0 个或多个）。
- ev.isDone() 拦截 [DONE] 或 message_stop——这类事件不进入 dispatch。

步骤 2.3：dispatch 做 JSON 解析

```
bool dispatch(const SSEEvent& ev) {
    if (ev.data.empty()) return true;
    json j = json::parse(ev.data, nullptr, false);  // 非抛出版本
    if (j.is_discarded()) return true;              // 坏帧跳过
    return dispatchOpenAI(j);
}
```

步骤 2.4：dispatchOpenAI 累积工具调用增量

```
// ---- ② ★ L4 新增：工具调用增量 ----
if (d.contains("tool_calls") && d["tool_calls"].is_array()) {
    for (const auto& tc : d["tool_calls"]) {
        const int idx = tc.value("index", 0);
        ToolBuf& buf = toolBufs[idx];
if (tc.contains("id") && !tc["id"].is_null())
            buf.id = tc["id"].get<std::string>();
if (tc.contains("function") && tc["function"].is_object()) {
            const auto& f = tc["function"];
            if (f.contains("name") && !f["name"].is_null()) {
                buf.name = f["name"].get<std::string>();
                keep = emit(ToolCallStart{buf.id, buf.name}) && keep;
            }
            if (f.contains("arguments") && !f["arguments"].is_null()) {
                const std::string piece = f["arguments"].get<std::string>();
                buf.raw += piece;            // ★ 只累积，不解析
                keep = emit(ToolCallDelta{buf.id, piece}) && keep;
            }
        }
    }
}
```

```
响应块（API 逐块返回的 chat.completion.chunk）
├─ choices: [                          → 候选结果列表
│    └─ choice: {                      → 单个候选（数组元素）
│         delta: {                     → 增量数据
│              content: "..."          → 正文增量
│              tool_calls: [...]       → 工具调用增量
│         },                           │
│         finish_reason: "stop"|...    → 停止原因
│         index: 0                     → 候选编号
│    }                                 │
│  ]                                   │
└─ usage: { ... }                      → 用量（仅最后一帧）
```

- key 用 index 而不是 id：一个 assistant 消息里可以有多个并行 tool_call，index 标识"第几个"。id 和 name 可能只在第一帧出现。
- 只累积，不解析：buf.raw += piece; 是唯一操作。中途任何位置 json::parse 都会失败（半个 JSON），而失败信息看起来像"模型给了非法参数"——会让分析者怀疑模型而不是怀疑时序。
- ToolCallStart 在 name 出现时 emit，ToolCallDelta 每次参数片段都 emit。

步骤 2.5：finish_reason 出现时 parse 一次

```
if (ch.contains("finish_reason") && !ch["finish_reason"].is_null()) {
    finishReason = fromOpenAIFinish(ch["finish_reason"].get<std::string>());
// ★ 只在这一刻 parse 一次 —— 此时 arguments 才是完整 JSON。
    for (auto& [idx, buf] : toolBufs) {
        if (buf.id.empty()) continue;
        json args = json::parse(buf.raw.empty() ? "{}" : buf.raw, nullptr, false);
        if (args.is_discarded()) {
            // 模型幻觉出了非法 JSON —— 容错成空对象
            args = json::object();
        }
        calls.push_back(ToolCall{buf.id, buf.name, std::move(args)});
        keep = emit(ToolCallDone{buf.id, calls.back().args}) && keep;
    }
    toolBufs.clear();
    keep = emit(DoneChunk{finishReason}) && keep;
}
```

值	含义
`"stop"`	模型到达自然停止点，或命中了 `stop` 序列
`"length"`	达到 `max_tokens` 限制，**回答被截断**
`"tool_calls"`	模型发起了工具调用
`"content_filter"`	内容被安全过滤器拦截
`"function_call"`	（已废弃）模型调用了函数

- 只在 finish_reason 出现时 parse——这是整个流程最关键的时序纪律。
- 容错：json::parse 失败时设成空对象，让参数校验去报错。模型下一轮能看到"缺少必
- finish_reason 必须放最后：一个 chunk 可能同时带 content、tool_calls 和 finish_reason。finish_reason 会被吞掉。

```
{"choices":[{"delta":{"tool_calls":[{
  "index": 0,
  "id": "call_abc123",
  "function": {"name": "Read", "arguments": ""}
}]},"finish_reason":null}]}

// chunk 2: arguments 片段
{"choices":[{"delta":{"tool_calls":[{
  "index": 0,
  "function": {"arguments": "{\"path\":"}
}]},"finish_reason":null}]}

// chunk 3: arguments 继续
{"choices":[{"delta":{"tool_calls":[{
  "index": 0,
  "function": {"arguments": "\"a.txt\"}"}
}]},"finish_reason":null}]}

// chunk 4: finish_reason 出现
{"choices":[{"delta":{},"finish_reason":"tool_calls"}]}
```

步骤 2.6：run 构造 ChatResponse，挂回 toolCalls

```
if (sctx) {
    for (auto& ev : sctx->parser.flush()) sctx->dispatch(ev);
ChatResponse resp;
    resp.usage           = sctx->usage;
    resp.message.role    = Role::Assistant;
    resp.message.content = sctx->textAcc;
    // ★ 工具调用必须挂回 message —— 下一轮要把它原样发回去
    resp.message.toolCalls = sctx->calls;
    // 有工具调用时 finish_reason 一律归一为 ToolUse
    resp.finishReason = sctx->calls.empty() ? sctx->finishReason
                                            : FinishReason::ToolUse;
    resp.message.finishReason = resp.finishReason;
    return resp;
}
```

- toolCalls 必须挂回 message：下一轮请求要把它原样发回去。如果丢了，模型会重复调用同一个工具——它不知道自己已经
- finish_reason 归一化：有些网关在带 tool_calls 时仍然回 "stop"，不能信它。只要有 calls，就强制 ToolUse。

```
请求: { stream: true, tools: [...], tool_choice: "auto",
        stream_options: { include_usage: true } }
  |
  ▼
chunk 1  { choices:[{ delta:{ role:"assistant", content:"" },
                      finish_reason:null }] }
  |      → 首帧空 delta，只带 role
  ▼
chunk 2  { choices:[{ delta:{ content:"我需要读取文件" },
                      finish_reason:null }] }
  |      → 正文增量
  ▼
chunk 3  { choices:[{ delta:{ tool_calls:[{ index:0,
                      id:"call_1", function:{ name:"Read",
                      arguments:"" } }] },
                      finish_reason:null }] }
  |      → 工具调用开始
  ▼
chunk 4  { choices:[{ delta:{ tool_calls:[{ index:0,
                      function:{ arguments:"{\"path\":" } }] },
                      finish_reason:null }] }
  |      → 参数片段
  ▼
chunk 5  { choices:[{ delta:{ tool_calls:[{ index:0,
                      function:{ arguments:"\"a.txt\"}" } }] },
                      finish_reason:null }] }
  |      → 参数片段结束
  ▼
chunk 6  { choices:[{ delta:{}, finish_reason:"tool_calls" }] }
  |      → 停止原因出现，此时才能 parse 工具参数
  ▼
chunk 7  { choices:[], usage:{ prompt_tokens:42,
             completion_tokens:128, total_tokens:170 } }
  |      → usage 块（choices 为空）
  ▼
data: [DONE]
```

步骤 2.7：main.cpp 把 resp.message 完整存回历史

```
totalUsage.inputTokens  += resp.usage.inputTokens;
totalUsage.outputTokens += resp.usage.outputTokens;
// ★ 把完整的 resp.message 存回历史（不能只存 content —— 工具字段会丢）
history.push_back(resp.message);
```

细节：这是"历史侧"的关键。resp.message 含 toolCalls，下一轮 toRequest 时会序列化成 assistant 消息带 tool_calls 字段。

阶段 3：派发侧 —— 五步管线执行工具

步骤 3.1：main 检测到 toolCalls 非空

```
// ---- 模型没有要工具 → 任务结束 ----
if (resp.message.toolCalls.empty()) {
    finalText = resp.message.content;
    break;
}
// ---- 模型要工具 → 执行并回喂 ----
for (const auto& tc : resp.message.toolCalls) {
```

```
宿主 main.cpp
  |
  | gpvHf8("AF_PLUGIN", "") → "C:\\...\\example_plugin.dll"
```

```
    finalText = resp.message.content;
    break;
}
// ---- 模型要工具 → 执行并回喂 ----
for (const auto& tc : resp.message.toolCalls) {
    ++toolCallCount;
    ToolResult tr = registry.dispatch(tc, *ui);
    std::cout << (tr.isError ? "[工具失败] " : "[工具返回] ")
              << tr.content.substr(0, 300)
              << (tr.content.size() > 300 ? " ..." : "") << "\n\n" << std::flush;
    history.push_back(tr.toMessage());
}
```

- toolCalls.empty() 是循环退出条件：模型不再请求工具，任务结束。
- 每个 ToolCall 单独 dispatch，结果单独回填。

步骤 3.2：dispatch 五步管线（第 0 步：查找）

```
ToolResult ToolRegistry::dispatch(const ToolCall& call, IUIPort& ui) {
    const Entry* entry = nullptr;
    for (const auto& e : entries_)
        if (e.def.name == call.name) { entry = &e; break; }
// ---- 0. 未知工具：把可用工具列表一起回喂 ----
    if (!entry) {
        std::string names;
        for (const auto& e : entries_) {
            if (!names.empty()) names += ", ";
            names += e.def.name;
        }
        return ToolResult::error(
            call.id, "未知工具 `" + call.name + "`。可用工具: " + names);
    }
    ...
}
```

步骤 3.3：第 1 步：Schema 校验

```
auto v = validate(call.args, entry->def.inputSchema);
if (!v)
    return ToolResult::error(call.id, "参数不合法: " + v.error + "\n请修正参数后重试。");
```

细节：

- validate 只检查 required 和 type，不是完整的 draft-07 validator。
- 错误信息是给模型看的（"缺少必需字段 path"），不是给程序员看的（"instance failed to validate against schema"）。

步骤 3.4：第 2 步：权限判定

```
if (gate_) {
    Decision d = gate_->evaluate(entry->def, call.args);
    if (d.kind == DecisionKind::Deny)
        return ToolResult::error(call.id, "被策略拒绝: " + d.reason);
    if (d.kind == DecisionKind::Ask && !ui.confirm(d.prompt))
        return ToolResult::error(
            call.id, "用户拒绝了这次操作。请换一种方式，或说明为什么必须要它。");
}
```

- DefaultPermissionGate 构造时用 stable_sort 把 deny 规则强制排到最前。
- deny 永远赢：即使规则表里 allow * 写在前面，构造后也会被 deny 规则覆盖。
- Ask 时调用 ui.confirm(prompt)，用户拒绝返回 false，工具返回错误信息给模型。

步骤 3.5：第 3 步：执行（invoke 分叉）

```
ToolResult ToolRegistry::invoke(const Entry& e, const json& args) {
    if (e.abi) {
        // ---- 插件路径 ----
        char* errOut = nullptr;
        char* raw = e.abi->execute(args.dump().c_str(), &errOut);
        if (!raw) {
            std::string msg = errOut ? errOut : "插件返回了未知错误";
            if (errOut && e.abi->freeString) e.abi->freeString(errOut);
            return ToolResult::error("", std::move(msg));
        }
        std::string content(raw);
        if (e.abi->freeString) e.abi->freeString(raw);
        return ToolResult::ok("", std::move(content));
    }
    if (e.fn) return e.fn(args);            // ← 内置函数路径
    return ToolResult::error("", "工具未实现");
}
```

- 两条路径互斥：e.abi != nullptr 走插件，e.fn 走内置。
- 插件路径是唯一的跨 ABI 边界：C ABI 三条铁律在此落地（extern "C"、不抛异常、谁分配谁释放）。
- 内置函数路径是 main.cpp 里 registerReadTool 注册的 lambda。

步骤 3.6：第 4 步：文本转换

```
r.content = sanitizeUtf8(r.content);
```

- 工具输出是外部字节：仓库里混着 GBK 源文件、命令输出带控制符。
- 不消毒的后果：下一轮请求 json::dump() 会抛 type_error.316。症状特别迷惑——"这一步明明成功了，下一轮却整个崩掉"。

步骤 3.7：第 5 步：输出过滤（大结果落盘）

```
r.callId = call.id;
return maybePersistLargeResult(std::move(r), entry->def, ".af-dump");
```

- 超过 persistThresholdChars 就写文件，上下文里只留约 2KB 预览 + 路径。
- Read 工具必须 persistThresholdChars = SIZE_MAX 豁免：否则会形成"读文件 → 落盘 → 再去读那个落盘文件"的死循环依赖。
- 落盘失败时退化成"截断 + 说明"，不让 turn 崩。

步骤 3.8：ToolResult::toMessage() 转成 Message

```
Message toMessage() const {
    Message m;
    m.role       = Role::Tool;
    m.toolCallId = callId;
    m.content    = content.empty() ? "(empty)" : content;
    return m;
}
```

- toolCallId 必须与产生它的 assistant.toolCalls[].id 配对。模型靠这个 id 匹配"哪个结果对应哪个调用"。
- 空串要替换掉：部分 provider 会拒绝 content="" 的 tool 消息。

```
data: [DONE]
```

```
宿主 main.cpp
  │
  │ envUtf8("AF_PLUGIN", "")  →  "C:\\...\\sample_plugin.dll"
  │
  ▼
ToolRegistry::loadPlugin(path, err)
  │
  ▼
PluginHandle::load(path, err)
  │
  ├─ LoadLibraryW("C:\\...\\sample_plugin.dll")   ← 用 W 版本处理中文路径
  │     └─ Windows 加载 DLL，运行 DllMain
  │
  ├─ GetProcAddress(lib, "af_plugin_entry")
  │     └─ 找到导出符号
  │
  ├─ fn()  →  &kPlugin                            ← 调用插件入口
  │
  ├─ 校验 p->abiVersion == AF_PLUGIN_ABI_VERSION   ← 关键防线
  │
  └─ p->init("{}")  →  0                          ← 初始化插件
  │
  ▼
返回 PluginHandle (含 lib_ 和 plugin_)
  │
  ▼
ToolRegistry 遍历 lib->tools()
  │
  ├─ 对每个 AFToolPlugin*，构造 Entry:
  │     e.def.name = t->name()                    ← "WordCount"
  │     e.def.description = t->description()
  │     e.def.inputSchema = json::parse(t->inputSchemaJson())
  │     e.def.risk = RiskLevel(t->riskLevel())
  │     e.def.timeoutMs = int(t->timeoutMs())
  │     e.def.persistThresholdChars = size_t(t->persistThresholdChars())
  │     e.abi = t
  │     e.pluginLib = lib                          ← ★ 持有库句柄，RAII
  │
  └─ entries_.push_back(e)
```

```
模型返回 tool_call: {name: "WordCount", args: {"path": "a.txt"}}
  │
  ▼
ToolRegistry::dispatch(call, ui)
  │
  ├─ 查找 Entry: e.def.name == "WordCount"
  │
  ├─ Schema 校验: validate(args, e.def.inputSchema)
  │     └─ 检查 required: ["path"] → 通过
  │
  ├─ 权限判定: gate_->evaluate(e.def, args)
  │     └─ 命中 Allow 规则 → 放行
  │
  ├─ invoke(e, args)
  │     └─ e.abi 非空 → 走插件路径
  │           │
  │           ├─ args.dump().c_str()  →  "{\"path\":\"a.txt\"}"
  │           │
  │           ├─ e.abi->execute(jsonStr, &errOut)
  │           │     └─ 插件内部: jsonStringField → "a.txt"
  │           │     └─ fopen / 逐字符统计
  │           │     └─ dupe("文件 a.txt: \n  行数   10\n...")
  │           │     └─ 返回 char* (插件分配)
  │           │
  │           ├─ raw 非空 → std::string content(raw)
  │           │
  │           ├─ e.abi->freeString(raw)              ← ★ 谁分配谁释放
  │           │
  │           └─ return ToolResult::ok("", content)
  │
  ├─ sanitizeUtf8(r.content)
  │
  └─ maybePersistLargeResult(r, e.def, ".af-dump")
        └─ persistThresholdChars == SIZE_MAX → 豁免，直接返回
```

步骤 3.9：history.push_back(tr.toMessage())

```
history.push_back(tr.toMessage());
```

此时 history 里已经有了：

- 上一轮的 assistant 消息（含 toolCalls）
- 这一轮的 tool 消息（工具结果）

阶段 4：回喂侧 —— 下一轮把工具结果发回模型

步骤 4.1：下一轮 for 循环开始

```
for (int round = 0; round < kMaxRounds; ++round) {
    ChatRequest req;
    req.system   = "...";
    req.messages = history;     // ★ history 现在含工具结果
    req.tools    = registry.defs();
    ...
}
```

步骤 4.2：toRequest 序列化三种消息形态

```
static json messageToJson(const Message& m) {
    json j{{"role", roleName(m.role)}, {"content", m.content}};
if (m.role == Role::Tool) {
        j["tool_call_id"] = m.toolCallId;
        return j;
    }
    if (!m.toolCalls.empty()) {
        json tcs = json::array();
        for (const auto& tc : m.toolCalls) {
            tcs.push_back({{"id", tc.id},
                           {"type", "function"},
                           {"function", {{"name", tc.name},
                                         {"arguments", tc.args.dump()}}}});  // 字符串！
        }
        j["tool_calls"] = tcs;
        if (m.content.empty()) j.erase("content");
    }
    return j;
}
```

三种形态：

形态	JSON 结构	谁产生
普通消息	{role, content}	system / user / 纯文本 assistant
assistant 带工具	{role, content, tool_calls:[...]}	上一轮模型的工具调用意图
tool 结果	{role:"tool", tool_call_id, content}	上一轮工具执行结果

1. arguments 必须是 JSON 字符串，不是对象。OpenAI 协议规定 function.arguments 是字符串类型。
2. 空 content 要 erase：assistant 只调工具没说话时，content 是空串，有些实现会拒绝。

步骤 4.3：模型看到工具结果，给出最终回答

服务端返回的下一个响应里，tool_calls 为空，content 是最终答案：

```
{"choices":[{"delta":{"content":"mock_target.txt 里有 3 个函数。"},"finish_reason":"stop"}]}
```

步骤 4.4：main.cpp 检测到 toolCalls 为空，任务结束

```
if (resp.message.toolCalls.empty()) {
    finalText = resp.message.content;
    break;    // ← 退出 for 循环
}
```

细节：break 跳出 for，进入收尾统计。循环的退出条件是"模型不再请求工具"

```
typedef struct AFToolPlugin {
    uint32_t abiVersion;

    const char* (*name)(void);
    const char* (*description)(void);
    const char* (*inputSchemaJson)(void);

    AFRisk (*riskLevel)(void);
    uint32_t (*timeoutMs)(void);
    uint64_t (*persistThresholdChars)(void);

    char* (*execute)(const char* argsJson, char** errOut);

    void (*freeString)(char*);
} AFToolPlugin;
```

```
typedef struct AFPlugin {
    uint32_t abiVersion;
    const char* (*id)(void);
    const char* (*version)(void);

    int (*init)(const char* configJson);
    void (*shutdown)(void);

    const AFToolPlugin* const* (*tools)(size_t* count);
} AFPlugin;
```

```
static char* wordcount_execute(const char* argsJson, char** errOut) {
    if (errOut) *errOut = nullptr;                    // ① 先清空 errOut

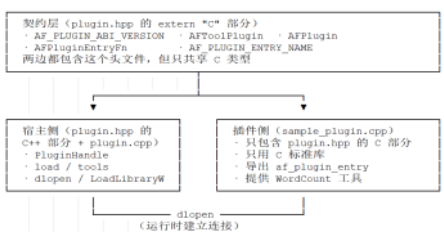
    const std::string path = jsonStringField(argsJson, "path");
    if (path.empty()) {
        if (errOut) *errOut = dupe("缺少必需字段 `path`");
        return nullptr;                               // ② 失败：NULL + errOut
    }

    std::FILE* f = std::fopen(path.c_str(), "rb");
    if (!f) {
        if (errOut) *errOut = dupe("无法打开文件: " + path);
        return nullptr;
    }

    std::size_t lines = 0, words = 0, chars = 0;
    bool inWord = false;
```

```
static const char* kName        = "WordCount";
static const char* kDescription = "统计文本文件的行数、词数与字符数。只读，安全。";
static const char* kSchema =
    R"({"type":"object","properties":{"path":{"type":"string",}"
    R"("description":"要统计的文件路径"}},"required":["path"]})";

static const AFToolPlugin kWordCountTool = {
    AF_PLUGIN_ABI_VERSION,
    []() -> const char* { return kName; },
    []() -> const char* { return kDescription; },
    []() -> const char* { return kSchema; },
    []() -> AFRisk { return AF_RISK_READ; },
    []() -> uint32_t { return 10000u; },
    []() -> uint64_t { return ~uint64_t(0); },   // 勤免落盘
    wordcount_execute,
    [](char* p) { std::free(p); },               // ★ freeString
};
```




```
class PluginHandle {
public:
    static std::optional<PluginHandle> load(const std::string& path, std::string& err);
    ~PluginHandle();

    PluginHandle(const PluginHandle&) = delete;
    PluginHandle& operator=(const PluginHandle&) = delete;
    PluginHandle(PluginHandle&&) noexcept;
    PluginHandle& operator=(PluginHandle&&) noexcept;

    const AFPlugin* plugin() const noexcept { return plugin_; }
    std::vector<const AFToolPlugin*> tools() const;
```

```
std::vector<const AFToolPlugin*> PluginHandle::tools() const {
```

```
static const AFToolPlugin* kToolTable[] = {&kWordCountTool};

static const char* plugin_id(void)      { return "sample-plugin"; }
static const char* plugin_version(void) { return "1.0.0"; }
static int  plugin_init(const char*)    { return 0; }   // 0 = 成功
static void plugin_shutdown(void)       {}

static const AFToolPlugin* const* plugin_tools(std::size_t* count) {
```

```
    PluginHandle(PluginHandle&&) noexcept;
    PluginHandle& operator=(PluginHandle&&) noexcept;

    const AFPlugin* plugin() const noexcept { return plugin_; }
    std::vector<const AFToolPlugin*> tools() const;

private:
    PluginHandle() = default;
    AfLibHandle lib_ = nullptr;
    const AFPlugin* plugin_ = nullptr;
};
```

```
std::vector<const AFToolPlugin*> PluginHandle::tools() const {
    std::vector<const AFToolPlugin*> out;
    if (!plugin_ || !plugin_->tools) return out;
    size_t n = 0;
    const AFToolPlugin* const* arr = plugin_->tools(&n);
    for (size_t i = 0; i < n; ++i) out.push_back(arr[i]);
    return out;
}
```

```
static const char* plugin_id(void)      { return "sample-plugin"; }
static const char* plugin_version(void) { return "1.0.0"; }
static int  plugin_init(const char*)    { return 0; }   // 0 = 成功
static void plugin_shutdown(void)       {}

static const AFToolPlugin* const* plugin_tools(std::size_t* count) {
    *count = sizeof(kToolTable) / sizeof(kToolTable[0]);
    return kToolTable;
}

static const AFPlugin kPlugin = {
    AF_PLUGIN_ABI_VERSION,
    plugin_id,
    plugin_version,
    plugin_init,
    plugin_shutdown,
    plugin_tools,
};
```

```
std::optional<PluginHandle> PluginHandle::load(const std::string& path, std::string& err) {
    PluginHandle h;

    // ① 打开动态库
#ifdef _WIN32
    h.lib_ = LoadLibraryW(utf8ToWide(path).c_str());
    if (!h.lib_) { err = "LoadLibrary 失败: " + std::to_string(GetLastError()); return std::nullopt;
}
    auto fn = reinterpret_cast<AFPluginEntryFn>(GetProcAddress(h.lib_, AF_PLUGIN_ENTRY_NAME));
#else
    h.lib_ = dlopen(path.c_str(), RTLD_NOW | RTLD_LOCAL);
    if (!h.lib_) { err = std::string("dlopen 失败: ") + (dlerror() ? dlerror() : ""); return std::nul
lopt; }
    auto fn = reinterpret_cast<AFPluginEntryFn>(dlsym(h.lib_, AF_PLUGIN_ENTRY_NAME));
#endif

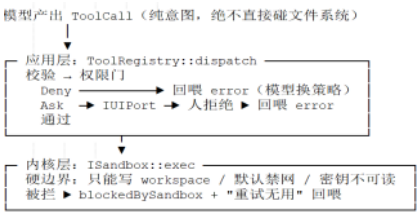
    // ② 找到导出符号
    if (!fn) { err = "未找到导出符号 " AF_PLUGIN_ENTRY_NAME; return std::nullopt; }

    // ③ 调用入口函数
    const AFPlugin* p = fn();
    if (!p) { err = "af_plugin_entry() 返回空"; return std::nullopt; }

    // ④ 校验 ABI 版本
    if (p->abiVersion != AF_PLUGIN_ABI_VERSION) {
        err = "ABI 版本不匹配: 插件=" + std::to_string(p->abiVersion) +
              " 宿主=" + std::to_string(AF_PLUGIN_ABI_VERSION);
        return std::nullopt;
    }
    h.plugin_ = p;

    // ⑤ 调用插件的 init
    if (p->init) {
        if (p->init("{}") != 0) { err = "插件 init 失败"; return std::nullopt; }
    }
    return h;
}
```

	权限门（Permission Gate）	沙箱（Sandbox）
回答的问题	这件事**该不该做**？	这件事**能造成多大破坏**？
所在层	应用层，内核之外	内核层（Landlock/seccomp/Seatbelt/Job Object）
判定依据	规则表 + 人的决定	文件系统/网络/进程的硬边界
失效方式	规则配错、模式匹配被绕过——**必然发生**	内核强制，绕不过
出错代价	弹了不该弹的窗、拦了不该拦的操作	无——它只在执行时兜底




```
ToolResult ToolRegistry::dispatch(const ToolCall& call, IUIPort& ui) {
    // 0. 未知工具 → 回喂可用工具列表
    // 1. Schema 校验（参数对不对）
    // 2. gate_->evaluate(...)            ← 权限在这里
    //      Deny → 回喂 error
    //      Ask  → ui.confirm(prompt)?  用户拒 → 回喂 error（附"换方式"指引）
    // 3. invoke(...)  执行
    // 4. sanitizeUtf8 + 超限落盘
}
```

```
class IPermissionGate {
public:
    virtual Decision evaluate(const ToolDef& tool, const json& args) const = 0;
};

class IUIPort {
public:
    virtual bool confirm(const std::string& prompt) = 0;    // 阻塞式询问
    virtual void notice(const std::string& msg) = 0;        // 单向通知
};
```

```
enum class DecisionKind { Allow, Ask, Deny };

struct Decision {
    DecisionKind kind = DecisionKind::Ask;
    std::string  reason;      // 给日志和模型看的理由
    std::string  prompt;      // 展示给用户的话（Ask 时弹窗用）
};

struct PermissionRule {
    DecisionKind kind = DecisionKind::Allow;
    std::string  toolPattern;   // "*" | "Bash" | "Bash(npm test)"
    std::string  reason;
};
```

```
std::vector<PermissionRule> rules{
    {DecisionKind::Deny,  "Bash(rm -rf *)", "删除类命令一律拒绝"},
    {DecisionKind::Allow, "Read",           "只读操作直接放行"},
};
```

```
Decision DefaultPermissionGate::evaluate(...) const {
    for (const auto& r : rules_) {          // deny 段 → 其余段 →
        if (r.matches(tool.name, args))
            return {r.kind, r.reason, describe(tool, args)};
    }
    return {fallback_, "无规则命中，走默认", describe(tool, args)};  // ← 兜底
}
```